

DEBACU – Modelo de Suscripciones y Control de Usuarios (Definitivo)

1) Objetivo (qué debe garantizar el sistema)

1. **El plan activo manda:** funcionalidades, límites de uso y **límite de usuarios**.
 2. **Nunca confiar en frontend:** todo control real se hace en Edge Functions + BD.
 3. **Transiciones seguras:** upgrade/downgrade/cancelación no pueden dejar el sistema en un estado “medio roto”.
 4. **Trazabilidad:** cada cambio relevante queda registrado (events + audit).
-

2) Planes y límites

2.1 Planes (regla dura)

Plan	Users max	Trial	Notas
FREE	1	30 días	Solo prueba, no se puede volver a FREE desde plan pago (recomendado)
BASIC	1	opcional	Pago recurrente
MEDIUM	2	opcional	Pago recurrente
PREMIUM	4	opcional	Pago recurrente

Decisión recomendada: FREE solo 1 vez por customer (trial) y luego bloqueado. Si permites volver a FREE, te comes abusos.

3) Estados de suscripción (máquina de estados)

Define estados claros (los mínimos que necesitas):

- **TRIAL_ACTIVE** (si FREE es trial real)
- **ACTIVE**
- **PENDING_PAYMENT** (checkout creado pero no pagado)
- **PAST_DUE** (si usas invoice/failed payments)
- **CANCELED**
- **SUSPENDED** (por fraude/abuso/admin)

Reglas:

- El sistema **solo habilita producto** si estado $\in \{\text{TRIAL_ACTIVE}, \text{ACTIVE}\}$

- **PENDING_PAYMENT** no habilita features premium (solo lectura / o acceso mínimo)
 - **SUSPENDED** = bloqueo total salvo facturación
-

4) Fuente de la verdad (Source of Truth)

4.1 La BD manda

- Tu tabla `subscriptions` (o `debacu_eval_subscriptions`) es la verdad operativa.
- Stripe es el “procesador de pagos”, pero **tu app no debe depender de llamadas live a Stripe** para decidir permisos.

4.2 Stripe sincroniza a BD (no al revés)

- Stripe webhook → actualiza BD
 - BD define permisos → Edge Functions aplican
-

5) Control de usuarios por plan (lo importante)

Aquí hay dos modelos. Te digo el que te conviene para Debacu.

Modelo recomendado: “Seats” por customer

- Cada usuario pertenece a un `customer_id`
- Hay un `users_active_count` calculable y una regla `max_users` según plan
- Si excede → se bloquean altas o se fuerza desactivación

Datos mínimos necesarios

- En tabla `users` (o tabla enlace):
 - `customer_id`
 - `status` (ACTIVE / DISABLED)
 - `role` (OWNER / ADMIN / USER...)
 - `created_at`

Regla dura de alta

Cuando se intenta crear o activar un usuario:

- contar usuarios **ACTIVE** del customer
- comparar con `max_users(plan)`
- si `count >= max` → rechazar

Esto debe hacerse:

- en Edge Function de **invitar usuario**
 - en Edge Function de **activar usuario**
 - opcionalmente también en BD con constraint/lógica (pero sin RPC; puedes hacerlo vía trigger si quieres)
-

6) Qué pasa en downgrade (donde muchos fallan)

Ejemplo: PREMIUM (4) → BASIC (1)

Tienes 2 estrategias. La “bonita” y la “realista”.

A) Estrategia realista (recomendada)

- Mantienes el plan nuevo activo
- Si usuarios activos > max:
 - El sistema entra en **LIMIT_EXCEEDED** (bandera interna)
 - **bloqueas acciones** sensibles hasta que reduzcan usuarios
 - Permitido: login del owner, gestión de usuarios, facturación
 - Bloqueado: crear evaluaciones, exportaciones, etc.

Ventaja: no apagas de golpe a gente.

Riesgo: hay que implementar “modo limitado”.

B) Estrategia agresiva (más simple)

- Al aplicar downgrade:
 - desactivas automáticamente usuarios sobrantes (los últimos creados)
 - solo dejas activo al OWNER + 0 más

Ventaja: implementación simple.

Riesgo: enfada a clientes (pero es clara).

Mi recomendación: B para empezar (simple y controlable). Luego evolucionas a A si lo necesitas.

7) Flujo Upgrade/Downgrade con Stripe

7.1 Upgrade inmediato (simple)

- Usuario solicita upgrade
- Edge Function crea Checkout/Portal (según estrategia)
- Stripe confirma pago (`checkout.session.completed`)

- Webhook actualiza:
 - subscription → ACTIVE
 - plan_id / price_id
 - current_period_end
- Desde ese instante:
 - max_users aumenta
 - habilitas features

7.2 Downgrade al fin de ciclo (recomendado)

- Usuario solicita downgrade
- Se programa para próximo ciclo (Stripe schedule o “cancel_at_period_end + nueva”)
- Tu BD marca:
 - required_plan_code
 - effective_date (next_billing_date)
- Al llegar la fecha:
 - webhook confirma
 - aplicas downgrade definitivo
 - aplicas regla de usuarios (A o B)

Esto evita líos contables.

8) Edge Functions mínimas (contratos claros)

Para que esto sea “definitivo”, necesitas estas funciones bien definidas:

1. **debacu_eval_subscription_get_active**
 - devuelve plan, estado, límites (max_users) y fechas
2. **debacu_eval_subscription_change_plan**
 - crea PENDING_PAYMENT + checkout session (o schedule)
3. **stripe_webhook_subscriptions**
 - aplica cambios reales a BD cuando Stripe confirma
4. **debacu_eval_users_invite_or_create**
 - valida seats antes de alta
5. **debacu_eval_users_set_status**
 - activar/desactivar usuario con validación de seats

9) Reglas anti-abuso (sin esto te van a explotar)

- FREE solo una vez por customer (trial).
 - Si cancelan y vuelven: vuelven a pago, no a FREE.
 - Limita creaciones por minuto (rate-limit simple por IP/user/customer).
 - Registra en `subscription_events` cada intento de cambio y su resultado.
-

10) Checklist de “Definitivo” (si esto se cumple, duermes tranquilo)

- Una sola suscripción “activa” por customer (constraint lógico)
- `max_users` calculado solo por plan activo
- Alta de usuarios siempre valida seats en backend
- Downgrade aplica regla clara (A o B)
- Todos los endpoints sensibles validan `ACTIVE/TRIAL_ACTIVE`
- Webhook es idempotente (no duplica eventos)
- Logs/audit de cambios de plan y usuarios